Search Box Documentation

1. How to run a test

- **Steps to run the test:**
    1. Download all dependencies
    2. Open your virtual environment to use pytest
    3. Call the test, from the “tests” folder so you can use the fixtures on conftest.py
        - pytest ./term_search_box/
        - pytest ./term_search_box/ --browser {Default/Safari/Edge/Chrome/Firefox}
        - pytest ./term_search_box/ {Browser Fixture} {added arguments}

See the pytest documentation on the read me for pytest specific information

- **Why the tests are structured like that:**

  The tests for this React Component are split into two tests, test_term_search_box.py and test_term_filter_buttons.py. They each test their respective sub-components that make up the functionality of the react component.

  test_term_search_box.py: This test file tests the functionality of the search box, the key of the component. Details on what tests are being performed are within the file.

  test_term_filter_buttons.py: This test file tests the functionality of the filter buttons, a supporting component. Details on what tests are being performed are within the file.

- **How to add more tests:**

  You can add more tests for this react component inside the folder “term_search_box”. Make sure to follow previously set guidelines for all component tests.

- **Areas for improvement:**

  There are areas for improvement regarding the test_term_filter_buttons.py, detailed in the file. The improvement would be better just for the security that all categories are functioning every time, whereas, now, only one category at random is tested.

  Also, testing the functionality of category expansion separately would be ideal as it is a supporting component as well.

---

2. How to call the `search_box` component

Usage / Example:
<SearchableDataViewer />

- **Why the search box is structured like that:**

  The search box was created to be a reusable component that could be called on any page. The structure is nested where separate files handle the creation of separate pieces of the entire component, see the names/preambles for specific details. There is also use of react based state handling to manage the different states of components, such as if a term is visible, a category is visible, or a state of a button.

  During the coding process we lost contact with our client due to a government shutdown, this meant we were unable to get the data that should be displayed, nor the libraries/formatting of the current FAA website. This means that the data is placeholder information grabbed from the AME guide, and that we decided not to create a backend.

  The decision not to create a backend meant that the data needed to be stored on the frontend, but be easily accessible and updateable. We achieved this using json documents, stored inside the “public/data” folder. This folder is accessible to the frontend during the loading of pages, whereas other parts of the code are not. As long as private/secured data is not stored here there is no currently known security threat.

- **Areas for improvement:**
  1. A backend could be added to improve the efficiency of the search box when there are >200 terms.
  2. The information should be reviewed and updated to be a better categorization
  3. If possible, moving the category of a term to be set up for the whole JSON rather than each term would be ideal.

2a. How to add more categories

- **Steps to add a category:**

  1. Create a new json file, with the name {INSERT_NAME}_terms.json
  2. Make the format match the others, such as a list of terms
  3. Add the filename with the category to the index.json file

- **Why categories are structured like that:**
  [Explain why the data is structured in arrays, objects, or with specific IDs.]

Categories are split into groups and the terms that fit into those groups. Each category is its own file to support a system similar to lazy-loading. This can be changed if going forward if desired, however this will need an overhaul that reduces performance.

- **Areas for improvement:**

  Searching makes all categories open if there is a term that matches the query, this would require a search button rather than “onChange” updates.

2b. How to add more terms

- **Steps to add a term:**

  1. Select the desired category
  2. Copy the format of the term above, except remove the comma if it is the last term, add one if not, in following with JSON formats
  3. Enter the data for the term
- **Why terms are structured like that:**

  Terms have five fields, the ID, name, description, med_cert_importance, and category. The ID is for ease of reference for developers, it can be removed if desired, it just means each term can be identified: category:ID. The rest are important for searching and viewing the data.

- **Areas for improvement:**

  As of now, there are no known improvements, besides what was mentioned above.